

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

835 · Image Overlap

LeetCode · Daily Question

Medium

Matrix

Hashing

1. What Type of Question Is This?

This is a **translation-matching** problem in disguise. Sliding an entire image by some (dx, dy) and counting overlapping 1's sounds like it needs you to try every possible shift and re-check the whole grid each time — but the real trick is noticing that **a shift only ever matters relative to actual 1-cells**. Any pair consisting of one 1-cell from `img1` and one 1-cell from `img2` implies exactly one specific displacement that would bring them into alignment. Counting how often each displacement is implied, across every such pair, tells you directly how many 1's would overlap under that shift — without ever simulating the translation on the full grid.

2. What Is Given?

- Two $n \times n$ binary matrices, `img1` and `img2`.
- You may **translate** `img1` by sliding it left/right/up/down by any number of units (a single shift in each direction, no rotation).
- Any 1-bits pushed outside the matrix border during the translation are simply erased (lost, not wrapped around).
- After translating, overlay `img1` on top of `img2` and count positions where **both** have a 1.

3. What Do We Need to Find?

The **largest possible overlap** — the maximum count of matching 1-positions achievable over every possible translation of `img1`.

Function Signature

■ C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  img1 = [[1,1,0],[0,1,0],[0,1,0]]
        img2 = [[0,0,0],[0,1,1],[0,0,1]]
Output: 3
```

Translating `img1` right by 1 unit and down by 1 unit shifts its three 1's into positions that all land on a 1 in `img2` — giving **3** overlapping positions, which turns out to be the maximum over every possible shift.

Example 2 — trivial 1×1 case

```
Input:  img1 = [[1]], img2 = [[1]]
Output: 1
```

Both single-cell images already have a 1 in the same (only) position, with no translation needed → **1**.

Example 3 — no 1's at all

```
Input:  img1 = [[0]], img2 = [[0]]
Output: 0
```

Neither image has any 1's, so no translation can ever create an overlapping 1 → **0**.

Constraints

- `n == img1.length == img1[i].length == img2.length == img2[i].length`
- $1 \leq n \leq 30$
- `img1[i][j]` and `img2[i][j]` are each either 0 or 1.

`n` is small (≤ 30), so even a fairly expensive approach will run comfortably here — but the pairwise-displacement technique below is the one that scales best, especially when the images are sparse (few 1's).

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Notice a shift is fully described by one vector.** Translating `img1` "left/right and up/down by any amount" is just picking a single (dx, dy) pair and adding it to every cell's coordinates. There's no rotation to worry about — every candidate translation is exactly one integer pair.
- **Step 2 — Realize most of the grid never matters.** Wherever `img1` has a 0, translating it there contributes nothing to the overlap count no matter what happens in `img2`. Only the positions that are already 1 in *both* images (after translation) count at all — so 0-cells can be ignored entirely.
- **Step 3 — Flip the question around.** Instead of asking "for this shift, how many 1's line up", ask "for this pair of one 1-cell from `img1` and one 1-cell from `img2`, what shift would bring them into alignment"? That shift is simply $(row2 - row1, col2 - col1)$.
- **Step 4 — Count how often each shift is implied.** Go through every pair of (1-cell in `img1`, 1-cell in `img2`), compute its implied shift, and tally it in a hash map keyed by (dx, dy) . A shift that's implied by k different pairs means translating `img1` by exactly that (dx, dy) makes all k of those pairs land on matching 1's simultaneously.
- **Step 5 — The answer is just the largest tally.** The maximum value in that hash map is the largest overlap achievable by any single translation — no need to ever explicitly simulate sliding the grid.

Which Algorithm Is Best?

Precompute the list of 1-cell coordinates in each image once, then iterate over every (cell from `img1`, cell from `img2`) pair, tallying implied shifts in a hash map and tracking the running maximum. This costs $O(k_1 \times k_2)$, where k_1 and k_2 are the counts of 1's in each image — often far smaller than the full n^2 grid size, and it never wastes time simulating shifts that don't align any 1's at all. The brute-force and "better" tiers below arrive here by first trying every possible shift directly against the whole grid, then narrowing the per-shift work down to just `img1`'s 1-cells, before finally flipping the loop order entirely to iterate over 1-cell pairs instead of shifts.

Approach 1 — Brute Force (Try Every Shift, Rescan the Whole Grid Each Time)

Idea: Literally try every possible (dx, dy) translation within the grid's range, and for each one, walk the entire $n \times n$ grid checking whether img1's (shifted) cell and img2's cell are both 1, counting matches as you go.

Steps

- Loop dx and dy each from $-(n-1)$ to $(n-1)$ — every translation that could possibly still overlap the grids at all.
- For each (dx, dy), scan every cell (r, c) of img1.
- Compute the shifted position (r+dx, c+dy); skip it if that falls outside the grid (translated-out 1's are erased).
- If both `img1[r][c]` and `img2[r+dx][c+dy]` are 1, count it as an overlap.
- Track the best count seen across every (dx, dy) tried.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        int best = 0;

        // Try EVERY possible shift, and for each one, rescan the ENTIRE grid.
        for (int dx = -(n - 1); dx <= n - 1; dx++) {
            for (int dy = -(n - 1); dy <= n - 1; dy++) {
                int count = 0;
                for (int r = 0; r < n; r++) {
                    for (int c = 0; c < n; c++) {
                        int nr = r + dx, nc = c + dy;
                        if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
                        if (img1[r][c] == 1 && img2[nr][nc] == 1) count++;
                    }
                }
                best = max(best, count);
            }
        }
        return best;
    }
};

int main() {
    vector<vector<int>> img1 = {{1,1,0},{0,1,0},{0,1,0}};
    vector<vector<int>> img2 = {{0,0,0},{0,1,1},{0,0,1}};
    Solution sol;
    cout << sol.largestOverlap(img1, img2) << endl;    // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def largestOverlap(self, img1, img2):
        n = len(img1)
        best = 0

        # Try EVERY possible shift, and for each one, rescan the ENTIRE grid.
        for dx in range(-(n - 1), n):
            for dy in range(-(n - 1), n):
                count = 0
```

```

        for r in range(n):
            for c in range(n):
                nr, nc = r + dx, c + dy
                if 0 <= nr < n and 0 <= nc < n:
                    if img1[r][c] == 1 and img2[nr][nc] == 1:
                        count += 1
            best = max(best, count)

    return best

if __name__ == "__main__":
    img1 = [[1,1,0],[0,1,0],[0,1,0]]
    img2 = [[0,0,0],[0,1,1],[0,0,1]]
    print(Solution().largestOverlap(img1, img2))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    largestOverlap(img1, img2) {
        const n = img1.length;
        let best = 0;

        // Try EVERY possible shift, and for each one, rescan the ENTIRE grid.
        for (let dx = -(n - 1); dx <= n - 1; dx++) {
            for (let dy = -(n - 1); dy <= n - 1; dy++) {
                let count = 0;
                for (let r = 0; r < n; r++) {
                    for (let c = 0; c < n; c++) {
                        const nr = r + dx, nc = c + dy;
                        if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
                        if (img1[r][c] === 1 && img2[nr][nc] === 1) count++;
                    }
                }
                best = Math.max(best, count);
            }
        }
        return best;
    }
}

const img1 = [[1,1,0],[0,1,0],[0,1,0]];
const img2 = [[0,0,0],[0,1,1],[0,0,1]];
console.log(new Solution().largestOverlap(img1, img2));    // Expected output: 3

```

Complexity: There are $O(n^2)$ possible shifts, and each one costs $O(n^2)$ to rescan the whole grid — $O(n^4)$ overall. With $n \leq 30$ that's about 810,000 operations, which is fast enough in practice here, but it wastes huge amounts of work rescanning 0-cells that could never contribute to the count.

Approach 2 — Better (Only Walk img1's 1-Cells for Each Shift)

Idea: Apply the Step 2 insight directly: precompute the list of img1's 1-cell coordinates once, then for each candidate shift, only check those cells (skipping every 0-cell entirely) instead of rescanning the whole grid.

Steps

- Collect every (r, c) where `img1[r][c] == 1` into a list, once, up front.
- For each candidate shift (dx, dy): walk only that list of 1-cells.
- For each 1-cell, compute its shifted position and check whether it's in bounds and equals 1 in img2; count it if so.
- Track the best count across every shift tried.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        vector<pair<int,int>> ones1;
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++)
                if (img1[r][c] == 1) ones1.push_back({r, c});

        int best = 0;

        // Still try every shift, but only walk img1's 1-cells instead of the whole grid.
        for (int dx = -(n - 1); dx <= n - 1; dx++) {
            for (int dy = -(n - 1); dy <= n - 1; dy++) {
                int count = 0;
                for (auto& [r, c] : ones1) {
                    int nr = r + dx, nc = c + dy;
                    if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
                    if (img2[nr][nc] == 1) count++;
                }
                best = max(best, count);
            }
        }
        return best;
    }
};

int main() {
    vector<vector<int>> img1 = {{1,1,0},{0,1,0},{0,1,0}};
    vector<vector<int>> img2 = {{0,0,0},{0,1,1},{0,0,1}};
    Solution sol;
    cout << sol.largestOverlap(img1, img2) << endl;    // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def largestOverlap(self, img1, img2):
        n = len(img1)
        ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]

        best = 0
```

```

# Still try every shift, but only walk img1's 1-cells instead of the whole grid.
for dx in range(-(n - 1), n):
    for dy in range(-(n - 1), n):
        count = 0
        for r, c in ones1:
            nr, nc = r + dx, c + dy
            if 0 <= nr < n and 0 <= nc < n and img2[nr][nc] == 1:
                count += 1
        best = max(best, count)

return best

if __name__ == "__main__":
    img1 = [[1,1,0],[0,1,0],[0,1,0]]
    img2 = [[0,0,0],[0,1,1],[0,0,1]]
    print(Solution().largestOverlap(img1, img2))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    largestOverlap(img1, img2) {
        const n = img1.length;
        const ones1 = [];
        for (let r = 0; r < n; r++)
            for (let c = 0; c < n; c++)
                if (img1[r][c] === 1) ones1.push([r, c]);

        let best = 0;

        // Still try every shift, but only walk img1's 1-cells instead of the whole grid.
        for (let dx = -(n - 1); dx <= n - 1; dx++) {
            for (let dy = -(n - 1); dy <= n - 1; dy++) {
                let count = 0;
                for (const [r, c] of ones1) {
                    const nr = r + dx, nc = c + dy;
                    if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
                    if (img2[nr][nc] === 1) count++;
                }
                best = Math.max(best, count);
            }
        }
        return best;
    }
}

const img1 = [[1,1,0],[0,1,0],[0,1,0]];
const img2 = [[0,0,0],[0,1,1],[0,0,1]];
console.log(new Solution().largestOverlap(img1, img2)); // Expected output: 3

```

Complexity: Still $O(n^2)$ possible shifts, but each one now costs $O(k_1)$ (the number of 1's in img1) instead of $O(n^2)$ — $O(n^2 \cdot k_1)$ overall. A real improvement whenever img1 is sparse, though it can still degrade to $O(n^4)$ if img1 is entirely 1's, and it's still built around "try every shift" rather than only the shifts that could actually matter.

Approach 3 — Optimal (Tally the Displacement Implied by Every Pair of 1-Cells)

Idea: Stop looping over candidate shifts altogether. Instead, loop over every pair (1-cell in img1, 1-cell in img2) directly, compute the exact shift that would bring that one pair into alignment, and tally it in a hash map. The largest tally, across all pairs, is the answer — every shift that's ever worth considering shows up naturally as a key in the map, and shifts that align nothing never get visited at all.

Steps

- Collect the 1-cell coordinates of both img1 and img2 into two lists.
- If either list is empty, no overlap is possible — return 0 immediately.
- For every (r1, c1) in img1's list and (r2, c2) in img2's list: compute the implied shift (r2 - r1, c2 - c1).
- Increment that shift's count in a hash map, and update the running best with the new count.
- Return the best count found once every pair has been processed.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        vector<pair<int,int>> ones1, ones2;
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++) {
                if (img1[r][c] == 1) ones1.push_back({r, c});
                if (img2[r][c] == 1) ones2.push_back({r, c});
            }

        if (ones1.empty() || ones2.empty()) return 0;

        unordered_map<long long, int> shiftCount;
        int best = 0;

        // For every pair of 1-cells, tally the ONE shift that would align them.
        for (auto& [r1, c1] : ones1) {
            for (auto& [r2, c2] : ones2) {
                int dx = r2 - r1, dy = c2 - c1;
                long long key = (long long)(dx + n) * 10000 + (dy + n); // pack (dx, dy)
                int cnt = ++shiftCount[key];
                best = max(best, cnt);
            }
        }
        return best;
    }
};

int main() {
    vector<vector<int>> img1 = {{1,1,0},{0,1,0},{0,1,0}};
    vector<vector<int>> img2 = {{0,0,0},{0,1,1},{0,0,1}};
    Solution sol;
    cout << sol.largestOverlap(img1, img2) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
from collections import defaultdict
```

```

class Solution:
    def largestOverlap(self, img1, img2):
        n = len(img1)
        ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
        ones2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]

        if not ones1 or not ones2:
            return 0

        shiftCount = defaultdict(int)
        best = 0

        # For every pair of 1-cells, tally the ONE shift that would align them.
        for r1, c1 in ones1:
            for r2, c2 in ones2:
                shift = (r2 - r1, c2 - c1)
                shiftCount[shift] += 1
                best = max(best, shiftCount[shift])

        return best

if __name__ == "__main__":
    img1 = [[1,1,0],[0,1,0],[0,1,0]]
    img2 = [[0,0,0],[0,1,1],[0,0,1]]
    print(Solution().largestOverlap(img1, img2))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    largestOverlap(img1, img2) {
        const n = img1.length;
        const ones1 = [], ones2 = [];
        for (let r = 0; r < n; r++)
            for (let c = 0; c < n; c++) {
                if (img1[r][c] === 1) ones1.push([r, c]);
                if (img2[r][c] === 1) ones2.push([r, c]);
            }

        if (ones1.length === 0 || ones2.length === 0) return 0;

        const shiftCount = new Map();
        let best = 0;

        // For every pair of 1-cells, tally the ONE shift that would align them.
        for (const [r1, c1] of ones1) {
            for (const [r2, c2] of ones2) {
                const key = (r2 - r1) + "," + (c2 - c1);
                const cnt = (shiftCount.get(key) || 0) + 1;
                shiftCount.set(key, cnt);
                if (cnt > best) best = cnt;
            }
        }
        return best;
    }
}

const img1 = [[1,1,0],[0,1,0],[0,1,0]];
const img2 = [[0,0,0],[0,1,1],[0,0,1]];
console.log(new Solution().largestOverlap(img1, img2)); // Expected output: 3

```

Complexity: $O(k_1 \times k_2)$ time, where k_1 and k_2 are the number of 1's in `img1` and `img2` — at most $O(n^4)$ if both images are entirely 1's (matching the brute-force bound in that pathological case), but far faster whenever either image is sparse, since only pairs of actual 1-cells are ever examined. $O(k_1 \cdot k_2)$ extra space in the worst case for the hash map of shifts.

Side-by-Side Comparison

1. Brute Force	Try every shift; rescan the whole grid for each one	$O(n^4)$	$O(1)$
2. Better	Try every shift; only walk img1's 1-cells for each one	$O(n^2 \cdot k_1)$	$O(k_1)$ for the 1-cell list
3. Optimal	Tally the implied shift for every (img1 1-cell, img2 1-cell) pair	$O(k_1 \cdot k_2)$	$O(k_1 \cdot k_2)$ worst case for the

Key Takeaways to Remember

- **"Slide one grid over another and count matches"** is a strong signal for the pairwise-displacement trick: every pair of "interesting" cells (here, 1-cells) implies exactly one shift, and tallying implied shifts finds the best one without ever simulating the slide.
- When most of a grid is irrelevant to the answer (here, every 0-cell), **precompute the short list of cells that actually matter** and iterate over that instead of the full grid.
- Flipping a nested loop's outer structure — from "for each shift, check relevant cells" to "for each relevant pair, derive its shift" — is a general pattern whenever the set of things you'd loop over (all shifts) is much larger than the set of things that could actually produce a nonzero contribution (pairs of 1-cells).
- The worst-case complexity of the optimal approach still degrades to $O(n^4)$ if both grids are entirely 1's — the technique doesn't change the theoretical worst case, but it adapts automatically to sparser, more realistic inputs where the brute-force tiers can't.
- All three tiers were validated against **Example 1** (img1 = [[1,1,0],[0,1,0],[0,1,0]], img2 = [[0,0,0],[0,1,1],[0,0,1]] → 3) across C++, Python, and JavaScript, and checked against Examples 2 and 3 as well.